Agent Tool-Use Guardrail Monitor (ATGM) – Course Project Design

Topic T7 · OST + SM joint project · Supervisor: Oumaima Araar · Team: Maxim Aleshkov, Asmar Baghirova, Agnes Biborka, Fanni Orosz, Áron Papp Balázs · October 2026

1. System architecture

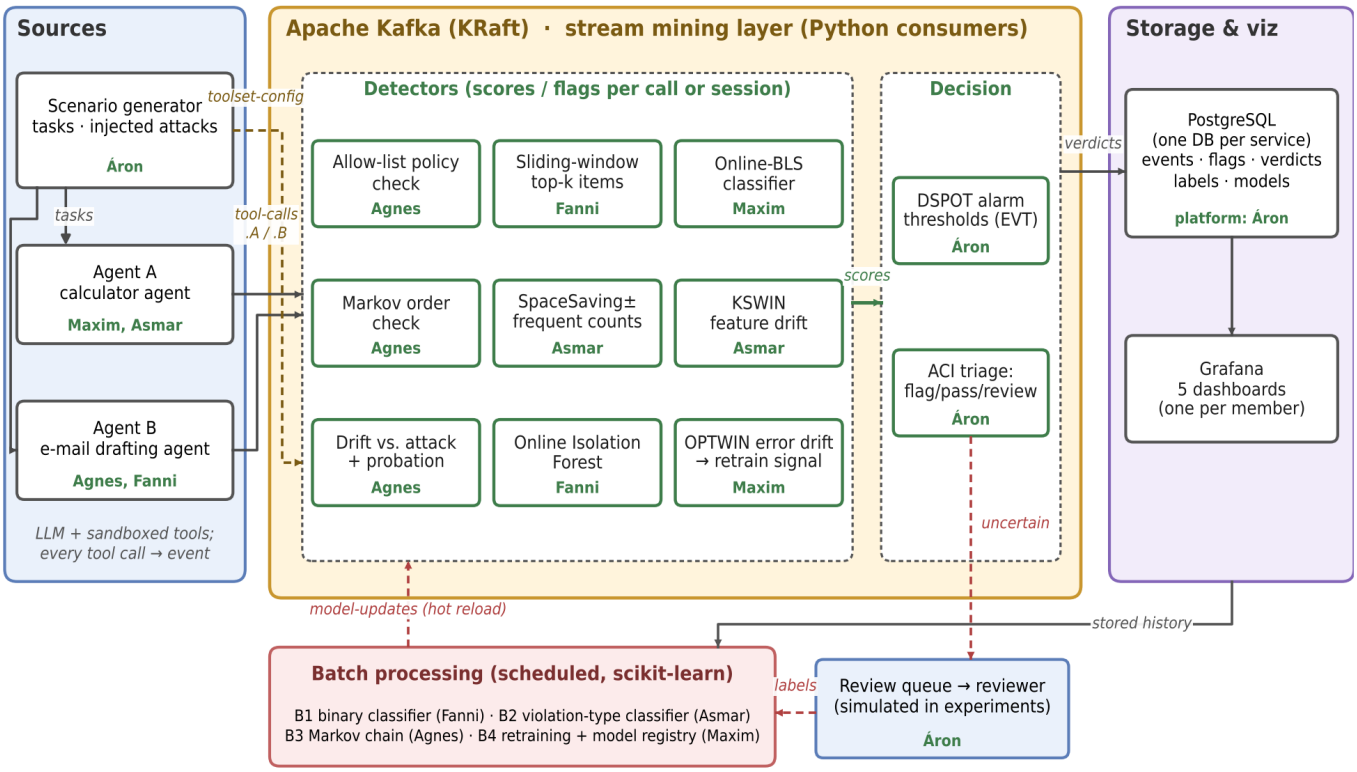


Figure 1 – ATGM architecture. Solid arrows: data flow (arrows inside the yellow area are Kafka topics); dashed red arrows: review and retraining loop. Green names: owner of each component.

2. System description

Sources and datasets. Two LLM agents with different toolsets run with sandboxed tools: **Agent A** is a calculator agent and **Agent B** an e-mail drafting agent; their exact toolsets are defined while they are built. A scenario generator sends them normal tasks and tasks containing prompt injections hidden in the task inputs, provoking unauthorized tool calls, unsafe arguments and out-of-order sequences. It also announces legitimate tool roll-outs on the *toolset-config* topic. The tool-call logs of the two agents are our two datasets; ground-truth labels are stored separately and never reach the detectors.

Streaming. Every tool call is published as a JSON event to Apache Kafka (*tool-calls.A / .B*), keyed by session id so that each session stays ordered. The stream mining layer consists of Python consumer services, one per team member. Detectors produce per-call or per-session scores and flags: allow-list and argument checks, a Markov order check, frequent-item counting for violation spikes, Online Isolation Forest anomaly scores, Online-BLS classification and drift detection on features (KSWIN) and on classifier errors (OPTWIN). The decision stage converts scores into alarms with a controlled false-alarm rate (DSPOT) and into flag / pass / review decisions (ACI). Uncertain sessions go to a review queue, answered by a simulated reviewer in the experiments; reviewed labels drive retraining.

Drift vs. attack safeguard. Detectors never change the rules or models themselves, so an attack that uses a newly added tool cannot be learned as normal. The allow-list changes only through trusted *toolset-config* events. A new tool is on probation for its first sessions (e.g. 50 sessions or 24 h): a sample of them is sent to review and all argument rules still apply. Models learn the new tool's behaviour only from sessions reviewed as normal, and an explained drift only lowers alert weights, never overrides a hard rule violation. The experiments include an attack that uses a new tool during its probation.

Storage, batch and visualization. PostgreSQL (one database per service) stores events, flags, verdicts, labels and model versions. Scheduled batch jobs train a binary classifier (B1), a violation-type classifier (B2) and a per-agent Markov chain of normal tool order (B3) on the stored data; B4 retrains them on reviewed labels and promotes a new model only if it beats the current one (champion/challenger). Promoted models are hot-reloaded by the stream services. Grafana shows one dashboard per member. All components run as Docker Compose services; a Kubernetes deployment is a stretch goal.

3. Planned stream mining tasks and algorithms

Member	Pipe-line	Algorithm and task in ATGM	Reference
Maxim Aleshkov	P2	**Online-BLS**: online classifier of session prefixes (compliant vs. violation)	Lei et al., *Sci. China Inf. Sci.* 69, 2026
	P6	**OPTWIN**: drift in the classifier's error stream, triggers retraining	Tosi & Theobald, ICDEW 2024
Fanni Orosz	P4	**Sliding-window top-k monitoring**: rolling violation counts per tool/agent, spike alerts	Chen et al., *Data Sci. Eng.* 2, 2017
	P2	**Online Isolation Forest**: unsupervised anomaly score per tool call	Leveni et al., ICML 2024
Asmar Baghirova	P4	**SpaceSaving±**: approximate counts of frequent tools, argument patterns, violation types	Zhao et al., PVLDB 2022
	P5	**KSWIN**: change detection on numeric features (argument length, failure rate, scores)	Raab et al., *Neurocomputing* 416, 2020
Agnes Biborka (2 of 3 to be selected)	P1	**Allow-list policy check**: real-time tool and argument rules on each call	OWASP LLM06:2025; Zhao et al., ClawGuard, arXiv 2026
	P3	**Markov order check**: tool sequence vs. learned multi-order Markov model (B3)	Sha et al., *IEEE TCC* 6(2), 2018; Boldt et al., *KAIS* 62, 2020
	P5	**Drift vs. attack**: separate legitimate behaviour drift from anomalous sequences	Li et al., arXiv:2403.03576, 2024
Áron Papp Balázs	P4	**DSPOT**: extreme-value-theory alarm thresholds for anomaly scores	Siffer et al., KDD 2017
	P6	**Adaptive Conformal Inference**: calibrated flag / pass / review triage under drift	Gibbs & Candès, NeurIPS 2021

P1–P6: pipelines of the topic description (P1 allow-list check, P2 ML classification, P3 order check, P4 rolling counts and alerts, P5 drift vs. attack, P6 review and retraining). Libraries: confluent-kafka, River, scikit-learn; each core algorithm is implemented by its owner.

Agents, batch and infrastructure ownership. Agent A: Maxim Aleshkov, Asmar Baghirova; Agent B: Agnes Biborka, Fanni Orosz (each agent pair also writes the normal and attack task templates for its agent). Batch: B1 binary classifier – Fanni Orosz; B2 violation-type classifier – Asmar Baghirova; B3 Markov chain – Agnes Biborka; B4 retraining and model registry – Maxim Aleshkov. Platform (Docker Compose, Kafka, PostgreSQL, Grafana), scenario generator engine and review queue with simulated reviewer – Áron Papp Balázs. Every member builds the Grafana dashboard for their own algorithms.